LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN
FLOWCHART DAN PEMROGRAMAN

Penyusun :

De'in Sutandi (201344007)
Iqbal Dzhurfan Hidayat (201344016)

1NK – D4 Teknik Telekomunikasi

Tanggal Penugasan: 23 Juli 2021

Tanggal Pengumpulan: 25 Juli 2021

Instruktur

Andry Haidar, ST., MT

Muhamad Yamin ST., MT

R. Wahyu Tri Hartono, DUTech., S.ST., M.Sc.



PROGRAM STUDI D4 – TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2021

I. TUJUAN

1. Memahami penggunaan algoritma dalam menyelesaikan permasalahan
2. Mampu membuat algoritma berupa flowchart
3. Memahami struktur dasar Pemrograman
4. Mengimplementasikan algoritma dalam Bahasa C/C++

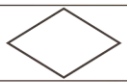
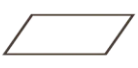
II. LANDASAN TEORI

Flowchart adalah suatu jenis diagram yang merepresentasikan algoritma atau langkah-langkah instruksi yang berurutan dalam suatu sistem. Flowchart biasanya digunakan sebagai bukti dokumentasi untuk menjelaskan gambaran logis sebuah sistem yang akan dibangun kemudian diberikan kepada programmer, dengan begitu, flowchart dapat membantu untuk memberikan solusi terhadap masalah yang bisa saja terjadi dalam membangun sistem. Pada dasarnya, flowchart digambarkan dengan menggunakan simbol-simbol. Setiap simbol mewakili suatu proses tertentu, adapun untuk menghubungkan satu proses ke proses selanjutnya selanjutnya digambarkan dengan menggunakan garis penghubung

Flowchart memiliki fungsi diantaranya adalah untuk menggambarkan, menyederhanakan suatu rangkaian proses atau prosedur sehingga dapat mudah dipahami dan mudah dilihat berdasarkan urutan langkah dari suatu proses.

Flowchart memiliki beberapa jenis, di antaranya :

1. Flowchart Sistem (System Flowchart)
2. Flowchart Flowchart Dokumen (Document Flowchart)
3. Flowchart Skematik (Schematic Flowchart)
4. Flowchart Program (Program Flowchart)
5. Flowchart Proses (Process Flowchart)

	**Flow Direction symbol** Yaitu simbol yang digunakan untuk menghubungkan antara simbol yang satu dengan simbol yang lain. Simbol ini disebut juga connecting line.
	**Terminator Symbol** Yaitu simbol untuk permulaan (start) atau akhir (stop) dari suatu kegiatan
	**Connector Symbol** Yaitu simbol untuk keluar - masuk atau penyambungan proses dalam lembar / halaman yang sama.
	**Connector Symbol** Yaitu simbol untuk keluar - masuk atau penyambungan proses pada lembar / halaman yang berbeda.
	**Processing Symbol** Simbol yang menunjukkan pengolahan yang dilakukan oleh komputer
	**Simbol Manual Operation** Simbol yang menunjukkan pengolahan yang tidak dilakukan oleh computer
	**Simbol Decision** Simbol pemilihan proses berdasarkan kondisi yang ada.
	**Simbol Input-Output** Simbol yang menyatakan proses input dan output tanpa tergantung dengan jenis peralatannya

	**Simbol Manual Input** Simbol untuk pemasukan data secara manual on-line keyboard
	**Simbol Preparation** Simbol untuk mempersiapkan penyimpanan yang akan digunakan sebagai tempat pengolahan di dalam storage.
	**Simbol Predefine Proses** Simbol untuk pelaksanaan suatu bagian (sub-program)/prosedure
	**Simbol Display** Simbol yang menyatakan peralatan output yang digunakan yaitu layar, plotter, printer dan sebagainya.
	**Simbol disk and On-line Storage** Simbol yang menyatakan input yang berasal dari disk atau disimpan ke disk.
	**Simbol magnetik tape Unit** Simbol yang menyatakan input berasal dari pita magnetik atau output disimpan ke pita magnetik.
	**Simbol Punch Card** Simbol yang menyatakan bahwa input berasal dari kartu atau output ditulis ke kartu
	**Simbol Dokumen** Simbol yang menyatakan input berasal dari dokumen dalam bentuk kertas atau output dicetak ke kertas.

Operator Relasi

Relational and Comparison operators atau dalam indonesia disebut sebagai "Operator Relasional dan Perbandingan" yang berfungsi untuk membandingkan dua *operand* atau ekspresi yang sebagai *operand*, hasil dari perbandingan akan menghasilkan nilai bertipe *Boolean*, "*True* atau *False*". Operator ini memiliki 6 bentuk operasi.

Operator	Makna
>	Lebih dari
>=	Lebih dari atau sama dengan
<	Kurang dari
<=	Kurang dari atau sama dengan
= =	Sama dengan
!=	Tidak sama dengan

Operator Logika

Operator logika biasa digunakan untuk menghubungkan ungkapan relasi yang terdapat dalam bahasa C

Operator	Maksud
&&	Dan (AND)
\|\| | Atau (OR) |
! | Negasi (NOT) |

Dengan menggunakan dua buah *operand*, Baik *operand1* maupun *operand2* dapat berupa ungkapan relasi maupun ungkapan logika. Hasil ungkapan mempunyai nilai benar atau salah, contoh hasil operasi ungkapan logika yang menggunakan operator && dan || untuk berbagai kemungkinan keadaan *operand.*

Operand1	Operand2	Hasil	
		||	&&
Salah	Salah	Salah	Salah
Salah	Benar	Benar	Salah
Benar	Salah	Benar	Salah
Benar	benar	Benar	Benar

Operator logika **ATAU** diwakili melalui lambang atau simbol ”||” akan menghasilkan nilai benar jika ada *operand* yang bernilai benar, dan akan menghasilkan nilai salah jika semua *operand* bernilai salah. Operator **DAN** diwakili melalui lambang atau simbol “ && ” akan memberikan hasil benar hanya jika kedua *operand* bernilai benar.

Fungsi Printf () (Continue)

- Tanda \ pada string yang dilewatkan sebagai argument prinf() mempunyai makna yang khusus , yaitu digunakan untuk menyatakan karakter khusus seperti karakter baris baru atau karakter backlash(miring kiri)
- Jadi karakter sebenarnya menyatakan sebuah karakter
- Contoh lain karakter yang ditulis dengan diawali tanda \ adalah :

Kode format	Arti
%c	Menampilkan sebuah karakter
%s	Menampilkan nilai string
%d	Menampilkan nilai desimal integer
%i	Menampilkan nilai desimal integer
%u	Menampilkan nilai desimal integer tak bertanda
%x	Menampilkan nilai hexadesimal integer
%o	Menampilkan nilai octal integer
%f	Menampilkan nilai pecahan
%e	Menampilkan nilai pecahan dalam notasi saintifik
%g	Pengganti %f atau %e tergantung yang terpendek
%p	Menampilkan suatu alamat memori untuk pointer

Perulangan

Struktur perulangan (atau dalam bahasa inggris disebut dengan loop) adalah instruksi kode program yang bertujuan untuk mengulang beberapa baris perintah.

Dalam merancang perulangan, setidaknya harus mengetahui 3 komponen berikut :

1. Kondisi **awal perulangan**
2. Kondisi pada **saat perulangan**
3. Kondisi yang harus dipenuhi agar **perulangan berhenti**

Berikut format dasar struktur perulangan *for* dalam Bahasa C :

```
for (start; condition; increment)
{
   // kode program
   // kode program
}
```

Start adalah kondisi pada saat awal perulangan. Biasanya kondisi awal ini berisi perintah untuk memberikan nilai kepada **variabel counter**. *Variabel counter* sendiri adalah sebuah variabel yang akan menentukan berapa banyak perulangan dilakukan. Kebanyakan programmer menggunakan variabel **i** sebagai variabel counter (ini tidak harus).

Condition adalah kondisi yang harus dipenuhi agar perulangan berjalan. Selama kondisi ini terpenuhi, maka *compiler* bahasa C akan terus melakukan perulangan. Misalnya condition ini berisi perintah **i < 5**, maka selama *variabel counter* **i** berisi angka yang kurang dari 5, maka lakukan perulangan.

Increment adalah bagian yang dipakai untuk memproses variabel counter agar bisa memenuhi kondisi akhir perulangan. Bagian ini akan selalu di eksekusi di setiap perulangan.

Disebut increment karena biasanya berisi operasi increment seperti i++, yang sama dengan i = i + 1. Maksudnya, dalam setiap perulangan naikkan variabel i sebanyak 1 angka. Namun kita juga bisa memberikan nilai lain, misalnya i = i + 2, sehingga variabel counter akan naik 2 angka setiap perulangan

Sebagai tambahan, terdapat istilah iterasi (iteration), yang berarti 1 kali perulangan. Istilah ini cukup sering dipakai ketika membahas tentang struktur perulangan.

Perulangan FOR

```
#include <stdio.h>

int main(void)
{
  int i;
  for (i = 1; i < 5; i++) {
    printf("Hello World \n");
  }
  return 0;
}
```

Gambar 2.1. Kode program Hello World perulangan FOR

Contoh kode program pada gambar 2 digunakan untuk menampilkan teks “Hello World”. Pada baris 5 terdapat sebuah variable I yang di set dengan tipe

data **integer**. Variabel ini nantinya akan digunakan sebagai **variable counter**, yakni yang menentukan kondisi akhir perulangan.

Perintah di baris 6, yakni **for (i = 1; i < 5; i++)**, bisa dibaca:

"Jalankan perulangan, mulai dari variabel **i = 1** sampai **i < 5**. Dalam setiap iterasi, naikkan nilai variabel i sebanyak 1 angka menggunakan perintah **i++**".

Output Program :

Hello World
Hello World
Hello World
Hello World

Output hanya menghasilkan 4 baris Hello World dikarenakan penggunaan tanda. Kondisi akhir perulangan adalah i < 5, yang artinya akan selalu bernilai **true** jika i kurang dari 5, tapi jika sudah sampai dengan 5 maka kondisi menjadi **false** dan perulangan berhenti.

Perulangan WHILE
Sebuah perulangan setidaknya memiliki 3 syarat: kondisi awal perulangan, kondisi pada saat perulangan, dan kondisi akhir perulangan.

Dalam perulangan FOR, ketiga syarat ini ditulis dalam 1 baris perintah, seperti: for (i = 1; i < 5; i++).

Di dalam perulangan WHILE, ketiga kondisi ini saling terpisah.

Berikut format dasar struktur perulangan WHILE dalam bahasa C:

```
1  start;
2  while (condition)
3  {
4     // kode program
5     // kode program
6    increment;
7  }
```

Gambar 2.2. Format dasar perulangan WHILE dalam Bahasa C

Di bagian start biasanya berupa perintah inisialisasi variabel counter, misalnya **i = 0.** Di bagian **condition** terdapat kondisi yang harus dipenuhi agar perulangan berjalan, misalnya **i < 5**. Kemudian perintah **increment** di dalam block perulangan yang di pakai untuk menaikkan nilai variabel counter, misalnya dengan perintah **i++.**

```c
#include <stdio.h>

int main(void)
{
  int i = 1;
  while (i <= 5){
    printf("Hello World \n");
    i++;
  }
  return 0;
}
```

Gambar 2.3. Kode program Hello World perulangan WHILE

Hasil kode program:

Hello World
Hello World
Hello World
Hello World
Hello World

Di baris 5 terdapat perintah untuk menginput angka 1 ke dalam variabel i. Nantinya, variabel i ini akan menjadi variabel counter yang dipakai untuk menentukan jumlah perulangan.

Proses perulangan di mulai di baris 6. Perintah while (i <= 5) artinya, selama nilai variabel i kurang atau sama dengan 5, maka jalankan perulangan.

Di dalam blok perulangan terdapat perintah printf(“Hello World \n”) di baris 7. Ini dipakai untuk menampilkan teks “Hello World“. Kemudian di baris 8 terdapat perintah increment, yakni i++. Perintah ini akan menaikkan nilai variabel i sebanyak 1 angka dalam setiap iterasi.

Perulangan while di atas akan di ulang sebanyak 5 kali, mulai dari i = 1, i = 2, i = 3, i = 4, hingga i = 5. Ketika nilai variabel counter i sudah mencapai 6, maka kondisi while (i <= 5) tidak terpenuhi lagi (false), sehingga perulangan berhenti.

Salah satu hal yang harus selalu diingat ketika membuat perulangan while adalah, jangan lupa membuat perintah increment. Jika tidak, kondisi akhir tidak akan pernah terpenuhi dan perulangan akan berjalan terus menerus. Ini dikenal dengan istilah infinity loop.

Perulangan DO WHILE

Perulangan **DO WHILE** merupakan modifikasi dari perulangan WHILE, yakni dengan memindahkan posisi pemeriksaan kondisi ke akhir perulangan. Artinya, kita akan lakukan dulu sebuah perulangan, baru di akhir diperiksa apakah kondisi variabel counter sudah terpenuhi atau belum.

Berikut format dasar struktur perulangan **DO WHILE** dalam bahasa C:

```
start;
do
{
 // kode program
 // kode program
 increment;
}
while (condition)
```

Gambar 2.4. Format dasar perulangan DO WHILE dalam Bahasa C

Sama seperti perulangan WHILE, di bagian start biasanya terdapat perintah inisialisasi variabel counter, misalnya i = 0. Kemudian di dalam block do ditulis kode program yang akan di ulang, tidak lupa sebuah perintah untuk menaikkan nilai variabel counter, misalnya dengan perintah i++.

Di bagian paling bawah, terdapat perintah while (condition). Di sinilah kondisi perulangan akan diperiksa. Selama kondisi ini menghasilkan nilai true, maka perulangan akan lanjut ke iterasi berikutnya.

```
#include <stdio.h>

int main(void)
{
  int i = 1;
  do {
    printf("Hello World \n");
    i++;
  }
  while (i <= 5);
  return 0;
}
```

Gambar 2.5. Kode program Hello World perulangan DO WHILE

Agar bisa dibandingkan dengan bahasan di perulangan FOR dan WHILE, gambar 2.5. merupakan contoh kode program bahasa C untuk menampilkan 5 kali teks ‘Hello World’ menggunakan perulangan DO WHILE:

Hasil kode program :

Hello World
Hello World
Hello World
Hello World
Hello World

Baris 5 mendefinisikan variabel counter i dan memberikan nilai awal 1.

Kemudian terdapat block kode program do di baris 6 – 9. Inilah kode program yang akan di ulang, yakni berisi sebuah perintah printf untuk menampilkan teks 'Hello World', dan sebuah perintah increment i++ untuk menaikkan nilai variabel i sebanyak 1 angka.

Terakhir di baris 10 terdapat perintah while (i <= 5). Artinya, selama variabel counter i bernilai kurang dari atau sama dengan 5, jalankan kembali perulangan.

Dalam perulangan DO WHILE ini kita juga bisa mengakses variabel counter, seperti contoh berikut:

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5    int i = 1;
6    do {
7      printf("%i ",i*3);
8      i++;
9    }
10   while (i <= 5);
11   return 0;
12 }
```

Gambar 2.6. Kode program variable counter perulangan DO WHILE

Output dari kode program diatas merupakan sebuah deret dengan kelipatan 3.

Output:
3 6 9 12 15

Array dalam Bahasa Pemrograman C

- Definisi Array

Array disebut juga larik, merupakan kelompok data dimana setiap elemen data menggunakan nama dan tipe data yang sama. Setiap komponen (elemen) array dapat dibedakan dan diakses melalui nomor indeksnya.

- Struktur array dapat digolongkan menjadi :

1. Array berdimensi satu
2. Array berdimensi dua atau banyak

- Array satu dimensi

Struktur array satu dimensi dapat dideklarasikan dengan bentuk umum sebagai berikut :

Tipe nama_var [*ukuran*];

- Tipe untuk menyatakan jenis elemen array (misalnya char, int, unsigned).
- ukuran DIGUNAKAN untuk menyatakan **jumlah maksimal** elemen array

contoh pendeklarasian suatu array yang memiliki **lima** elemen **bertipe float** adalah sebagai berikut:

float nilai_tes [5];

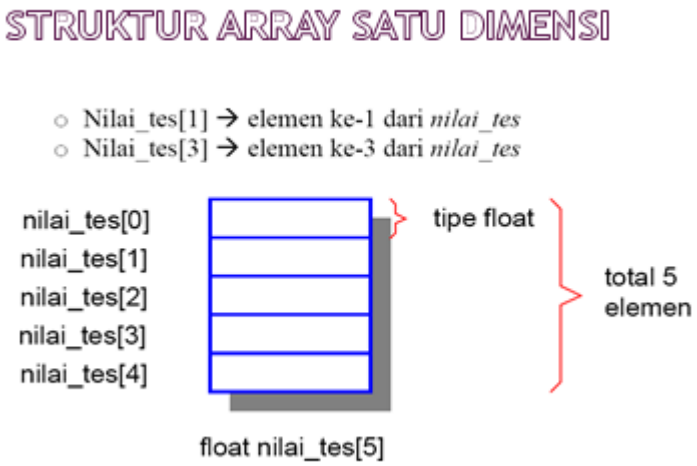
- Mengakses elemen array

Pada bahasa C, data array akan disimpan dalam memori pada lokasi yang berurutan.Indeks pada Elemen pertama selalu diawali dengan indeks ke-0.Jika pada contoh variabel nilai_tes mempunyai 5 elemen, maka elemen pertama mempunyai indeks sama dengan 0, elemen kedua mempunyai indeks 1, dan seterusnya hingga indeks 4.

Bentuk umum pengaksesan suatu elemen variabel array adalah:

Nama_var [indeks];

- Struktur array satu dimensi



- Memasukkan elemen array

Contoh untuk memasukkan data ke variabel array : nilai_tes[0] = 70; a. contoh 1

atau

scanf("%f", &nilai_tes[3]; à contoh 2

Contoh pertama merupakan pemberian nilai 70 ke elemen variabel nilai_tes ke-0.

Contoh kedua merupakan perintah untuk membaca data bilangan dari keyboard dan dimasukkan ke elemen nilai-tes yang ke-3.

Statemen *&nilai_tes[3]* menunjukkan "alamat dari elemen variabel nilai_tes yang ke-3".

- Contoh Program

```
1.  #include <stdio.h>
2.  #define maks_tes 5
3.  main()
4.  {
5.      int i;
6.      float nilai_tes[maks_tes];
7.
8.      for (i=0;i<maks_tes;i++)
9.      {
10.             printf("Nilai tes ke-%d: ",i+1);
11.             scanf("%f", &nilai_tes[i]);
12.     }
13.
14.      for (i=0;i<maks_tes;i++)
15.     {
16.             printf("Nilai tes ke-%d: %f\n",i+1,nilai_tes[i]);
17.     }
18. }
```

- Array 2 dimensi

Tipe data array dua dimensi biasa digunakan untuk menyimpan, mengolah maupun menampilkan suatu data dalam bentuk tabel maupun matriks.

- Mengakses array 2 dimensi

Untuk mengakses elemen array dua dimensi dapat dilakukan dengan statemen:

nama_variabel[indeks pertama][indeks kedua];

Sebagai contoh:

- jumlah_mhs[1][2] = 129;
- Merupakan instruksi untuk mengakses elemen array pada baris ke-1, kolom ke-2 dan mengisi dengan data bernilai '129'.

- Contoh Program

```
1.  #include <stdio.h>
2.  main()
3.  {
4.      int i,j;
5.      int matriks[4][3]; /* deklarasi array */
6.
7.      /* Membaca data dari keyboard */
8.      for (i=0;i<4;i++)
9.      {
10.         for(j=0;j<3;j++)
11.         {
12.           printf("matriks (%d, %d): ",i,j);
13.           scanf("%d", &matriks[i][j]);
14.         }
15.     }
16.     printf("\n\n");
```

- Inisialisasi array 2 dimensi

Deklarasi dan inisialisasi yang dilakukan berupa :

```
static int huruf_A[8][8] =
    { { 0, 1, 1, 1, 1, 1, 0, 0 },
      { 0, 1, 0, 0, 0, 1, 0, 0 },
      { 0, 1, 0, 0, 0, 1, 0, 0 },
      { 1, 1, 1, 1, 1, 1, 1, 0 },
      { 1, 1, 0, 0, 0, 0, 1, 0 },
      { 1, 1, 0, 0, 0, 0, 1, 0 },
      { 1, 1, 0, 0, 0, 0, 1, 0 },
      { 0, 0, 0, 0, 0, 0, 0, 0 },
    };
```

atau

```
static int huruf_A[8][8] =
    {  0, 1, 1, 1, 1, 1, 0, 0 ,
       0, 1, 0, 0, 0, 1, 0, 0 ,
       0, 1, 0, 0, 0, 1, 0, 0 ,
       1, 1, 1, 1, 1, 1, 1, 0 ,
       1, 1, 0, 0, 0, 0, 1, 0 ,
       1, 1, 0, 0, 0, 0, 1, 0 ,
       1, 1, 0, 0, 0, 0, 1, 0 ,
       0, 0, 0, 0, 0, 0, 0, 0 ,
    };
```

- Array dimensi banyak

Bahasa C memungkinkan untuk membuat struktur data array yang mempunyai dimensi lebih dari dua.

Bentuk umum pendeklarasian array berdimensi banyak adalah:

tipe nama_variabel[ukuran 1][ukuran 2] … [ukuran n];

Sebagai contoh :
int data_huruf[2][8][8];
Merupakan pendeklarasian array data_huruf sebagai array tiga dimensi

Proses inisialisasi pada array tiga dimensi sama halnya dengan array dimensi satu atau pun dua, Contohnya seperti pada gambar dibawah.

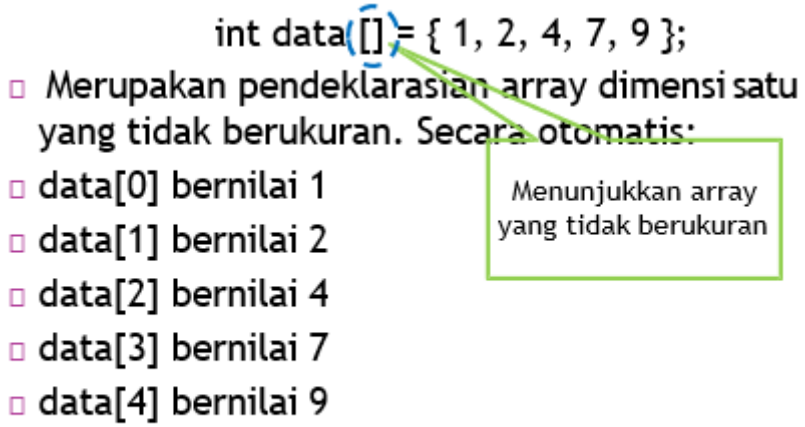
```
Static int data_huruf[2][8][8] =
{ { { 0, 1, 1, 1, 1, 1, 0, 0 },
    { 0, 1, 0, 0, 0, 1, 0, 0 },
    { 0, 1, 0, 0, 0, 1, 0, 0 },
    { 1, 1, 1, 1, 1, 1, 1, 0 },
    { 1, 1, 0, 0, 0, 0, 1, 0 },
    { 1, 1, 0, 0, 0, 0, 1, 0 },
    { 1, 1, 0, 0, 0, 0, 1, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0 },
  },
  { { 1, 1, 1, 1, 1, 1, 0, 0 },
    { 1, 0, 0, 0, 0, 1, 0, 0 },
    { 1, 0, 0, 0, 0, 1, 0, 0 },
    { 1, 1, 1, 1, 1, 1, 1, 0 },
    { 1, 1, 0, 0, 0, 0, 1, 0 },
    { 1, 1, 0, 0, 0, 0, 1, 0 },
    { 1, 1, 1, 1, 1, 1, 1, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0 },
  }
};
```

Inisialisasi aray **data_huruf** bertipe tiga dimensi.

- Inisialisasi array tak berukuran

Inisialisasi array yang tak berukuran dapat dilakukan untuk array berdimensi satu atau lebih.Untuk array berdimensi lebih dari satu, dimensi terkirilah yang boleh tidak berukuran.
Dengan cara ini maka tabel dalam array dapat diperluas atau dikurangi tanpa mengubah ukuran array.
Sebagai contoh:

int data[] = { 1, 2, 4, 7, 9 };



2.1 Fungsi Dalam Bahasa pemrograman C

- Definisi

Fungsi merupakan bagian (blok) dari kode program yang dirancang untuk melaksanakan tugas tertentu dengan tujuan:

a) Program menjadi terstruktur, sehingga mudah dipahami dan mudah dikembangkan.
b) Dapat mengurangi pengulangan kode (duplikasi kode).

Tugas khusus yang dilakukan suatu fungsi dapat dilihat dari beberapa fungsi standard, misalnya:

- getch( ), fungsi untuk membaca kode tombol papan ketik yang ditekan.
- printf( ), fungsi untuk menampilkan informasi atau data ke layer

Pada umumnya fungsi memerlukan masukan yang dinamakan sebagai **argumen atau parameter.** Masukan ini selanjutnya diolah oleh fungsi dan hasil keluarannya dapat berupa sebuah nilai (disebut nilai keluaran fungsi atau nilai balik fungsi).

- Bentuk umum fungsi

```
Penentu_tipe  nama_fungsi (daftar parameter)

Deklarasi parameter
{

  tubuh fungsi

}
```

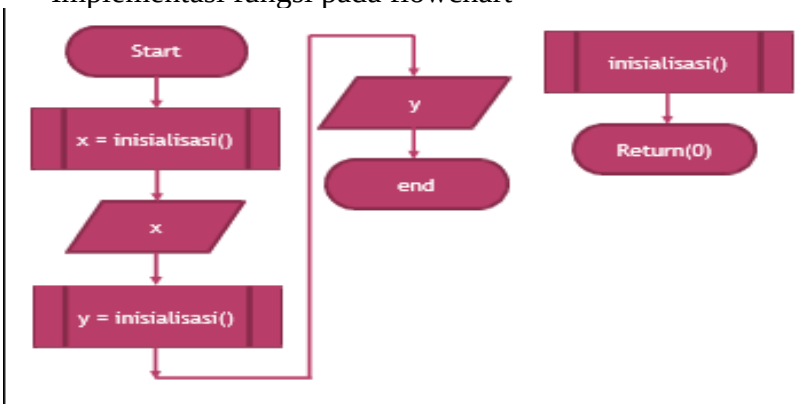
Penentu tipe digunakan untuk menentukan tipe keluaran fungsi, yang dapat dilipih dari salah satu tipe data yang berlaku dalam C, seperti *char* atau *int* dan lainnya. Jika penentu tipe tidak disebutkan, maka hasil keluaran akan dianggap sebagai *int*.

Sebuah fungsi dapat saja tidak mengandung parameter sama sekali, bahkan tanpa deklarasi parameter

- Contoh fungsi tanpa parameter

```
                    Nama funsi
                    Tanpa argumen
Inisialisasi( )  <- tidak ada tanda titik-koma
{                <- awal fungsi
   return(0);    <- tubuh fungsi
}                <- akhir fungsi
```

- Implementasi fungsi pada flowchart



- Nilai akhir (return)

Sebuah fungsi dapat mempunyai nilai keluaran dengan cara memberikan nilai keluaran pada *variable* yang terletak setelah pernyataan *return* atau menyisipkan nilai secara langsung setelah pernyataan *return*.

Contoh pada program fungsi menjelaskan bahwa suatu fungsi cukup didefinisikan satu kali, tetapi dapat digunakan beberapa kali untuk menghindari duplikasi kode dan menghemat penulisan program maupun penyimpanan kode dalam memori

- Fungsi tanpa nilai return

Penggunaan fungsi dapat dilakukan tanpa memberikan nilai akhir (return), yaitu dengan menambahkan pernyataan *void* di awal nama fungsi. seperti contoh berikut :

void *function_name* (tipe_data parameter)
{
.....
}

- Proptipe fungsi

- Pendefinisian fungsi diawal digunakan untuk menjelaskan kepada kompiler mengenai:
  - Tipe keluaran fungsi, nilai *return;*
  - Jumlah parameter;
  - Tipe dari masing-masing parameter.
- Bagi kompiler, informasi yang didefinisikan diawal fungsi digunakan untuk memeriksa keabsahan (validation code) sekaligus proses koreksi pada parameter dalam pemanggilan fungsi serta penamaan dan tipe data fungsi.

- Parameter formal dan actual

Parameter formal adalah variabel yang ada pada daftar parameter dalam definisi fungsi. Pada fungsi jumlah( ) Program_3, parameter x dan y dinamakan sebagai parameter formal.

Parameter aktual adalah parameter (tidak selamanya menyatakan variabel) yang digunakan dalam pemanggilan fungsi. Pada pernyataan:

c = jumlah(a, b);
y = jumlah(20.1, 0.9);

- Parameter a dan b merupakan parameter aktual dari fungsi jumlah (), demikian pula 20.1 dan 0.9 (meskipun bukan berupa variabel).

- Cara melewatkan parameter

1. Pada pemanggilan dengan nilai, nilai dari parameter aktual akan disalin ke parameter formal.
2. Dengan cara ini nilai parameter aktual tidak bisa berubah sekalipun nilai parameter formal berubah.
3. Pemanggilan dengan **referensi merupakan upaya untuk melewatkan alamat dari suatu variabel ke dalam fungsi.**

Cara ini dapat digunakan untuk mengubah isi suatu variabel di luar fungsi dengan pelaksanaan pengubahan dilakukan di dalam fungsi.

III. ALAT DAN KOMPONEN

1. Komputer atau Laptop
2. Compiler
3. Flowchart
4. Kertas
5. Pensil/bolpoint

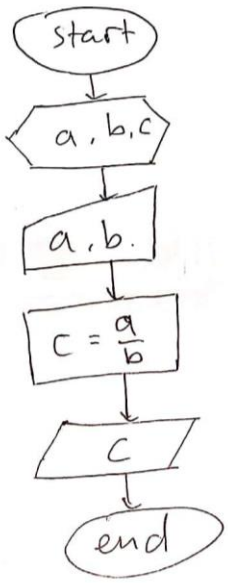
IV. LANGKAH KERJA

1. Persiapkan Semua Flowchart yang telah diberikan pada Mata Kuliah Teori Algoritma dan Pemograman
2. Lakukanlah analisis terlebih dahulu terhadap Flowchart-flowchart tersebut
3. Kemudian buatlah program dari flowchart tersebut
4. Setelah itu tampilkan hasil dari setiap program yang telah dibuat
5. Apabila flowchart dan programnya telah dibuat, lakukan analisis terhadap keduanya

V. TUGAS DAN PEMBAHASAN

Flowchart

Operasi pembagian



Analisis :

1. Mulai
2. Inisialisasi a, b, c
3. Inputkan a, b

4. Proseskan c= a/b
5. Cetak c
6. Selesai

Program dan hasil

```c
#include <stdio.h>

int main()
{
    int a,b,c;
    printf ("Masukan Nilai a = ");
    scanf("%d", &a);
    printf ("Masukan Nilai b = ");
    scanf("%d", &b);
    c=a/b;
    printf("%d", c);
}
```

```
C:\Users\DEIN S\Downloads\2.exe
Masukan Nilai a = 10
Masukan Nilai b = 2
5
--------------------------------
Process exited after 4.968 seconds with return value 1
Press any key to continue . . .
```

```c
#include <stdio.h>

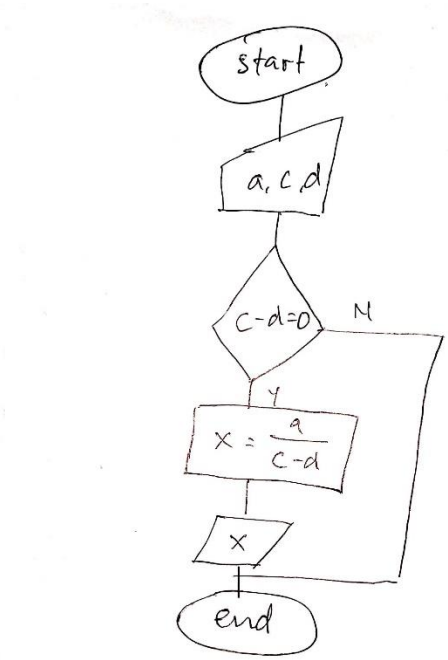
int main() {

    int a =4, b =2, c;
    c= a/b;
    printf ("%d", c);
}
```

```
E:\My Data\Kuliah\Semester 2\Algoritma Pemrograman\Praktik
2
--------------------------------
Process exited after 0.7555 seconds with return value 1
Press any key to continue . . .
```

Flowchart

Pengoperasian matematika $X = \frac{a}{c-d}$



Analisis :

1. mulai
2. inputkan a, c, d
3. cek kondisi c-d=0,
4. jika iya maka proses x=$\frac{a}{c-d}$, jika tidak maka langsung selesai
5. cetak x
6. selesai

Program dan hasil

```c
#include <stdio.h>

void main() {

    float a, c, d, x;;

    printf("Nilai a = ");
    scanf("%f", &a);

    printf("Nilai c = ");
    scanf("%f", &c);

    printf("Nilai d = ");
    scanf("%f", &d);

    x=a/(c-d);

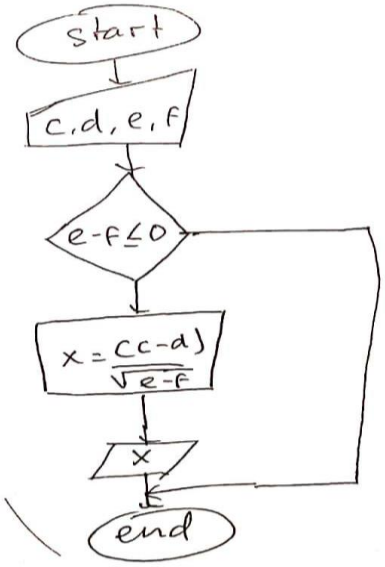
    if(c-d == 0){
        printf("%f", a);
    }

    else{
        printf("%1.f", x);
    }

}
```

```
E:\My Data\Kuliah\Semester 2\Algoritma Pemrograman\Praktik
Nilai a = 12
Nilai c = 8
Nilai d = 4
3
--------------------------------
Process exited after 11.81 seconds with return value 1
Press any key to continue . . .
```

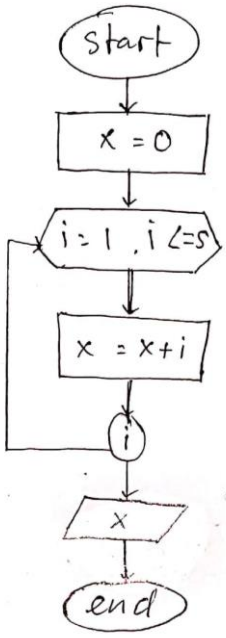
Flowchart



Analisis :

1. mulai
2. inputkan c, d, e, f
3. cek kondisi e-f<=0,
4. jika iya maka proses x=$\frac{c-d}{\sqrt{e-f}}$, jika tidak maka langsung selesai
5. cetak x
6. selesai

Flowchart



Analisis :

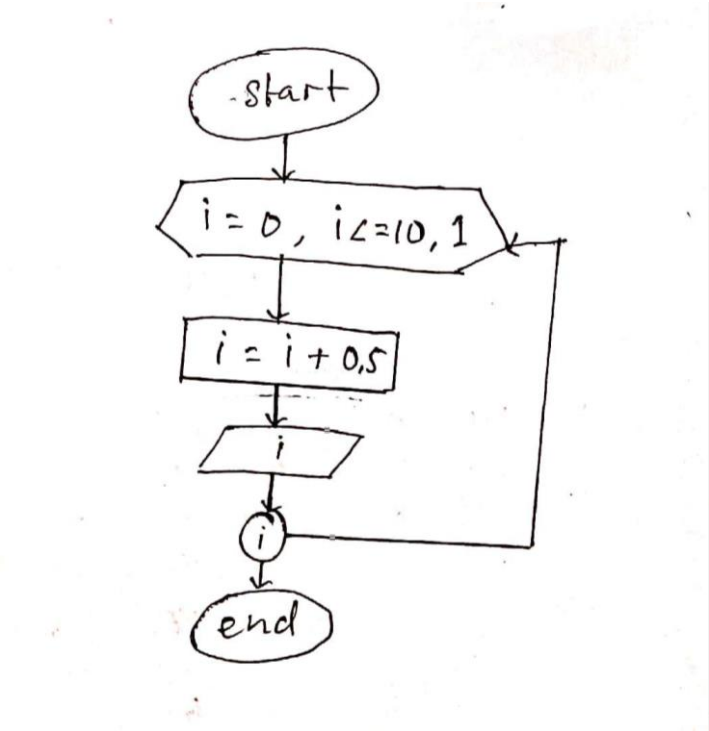
1. Mulai
2. Inisialisasi x=0, i=1, dan i<=5
3. Proses nilai x = x + i
4. Ulang proses sampai dengan i<=5
5. Cetak x
6. Selesai

Hasil Program

```
1   #include <stdio.h>
2
3   void main(){
4       int i,x;
5
6       x=0;
7       for (i=1; i<=5; i++) {
8           x=x+i;
9           printf(" %d",i);
10      }
11      printf("\nHasil Penjumlahan = %d", x);
12
13  }
```

```
E:\My Data\Kuliah\Semester 2\Algoritma Pemrograman\Praktiku
 1 2 3 4 5
Hasil Penjumlahan = 15
--------------------------------
Process exited after 0.3513 seconds with return value 23
Press any key to continue . . .
```

Flowchart



Analisis :

1. Mulai
2. Inisialisasi i=0, i<=10, dan increment 1
3. Proses nilai i = i + 0.5
4. Cetak nilai i
5. Ulangi proses sampai dengan i<=10
6. Selesai

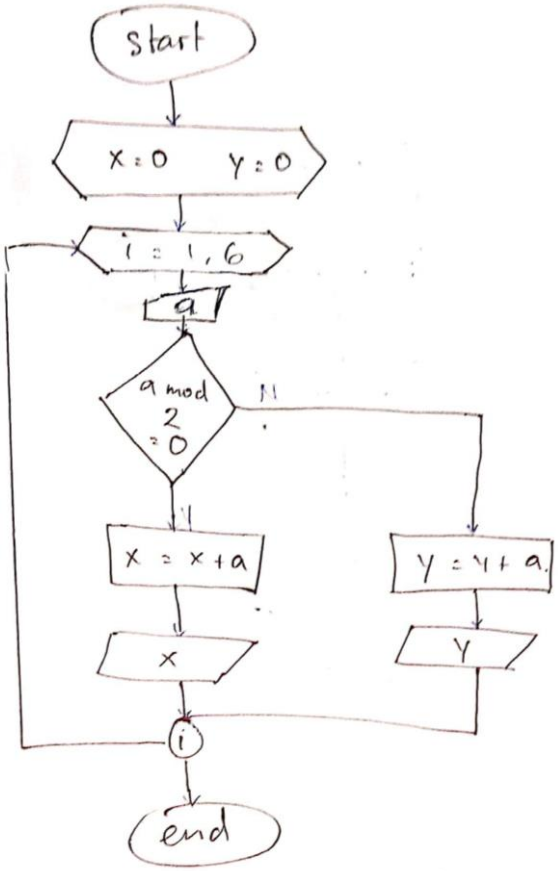
Program dan hasil

```
1   #include <stdio.h>
2
3   int main () {
4       float i;
5       for (i=0; i<=10; i=i+0.5) {
6       printf("%.1f \n",i);
7       }
8
9       return 0;
10  }
```

```
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
8.5
9.0
9.5
10.0

--------------------------------
Process exited after 0.0203 seconds with return value 0
Press any key to continue . . .
```

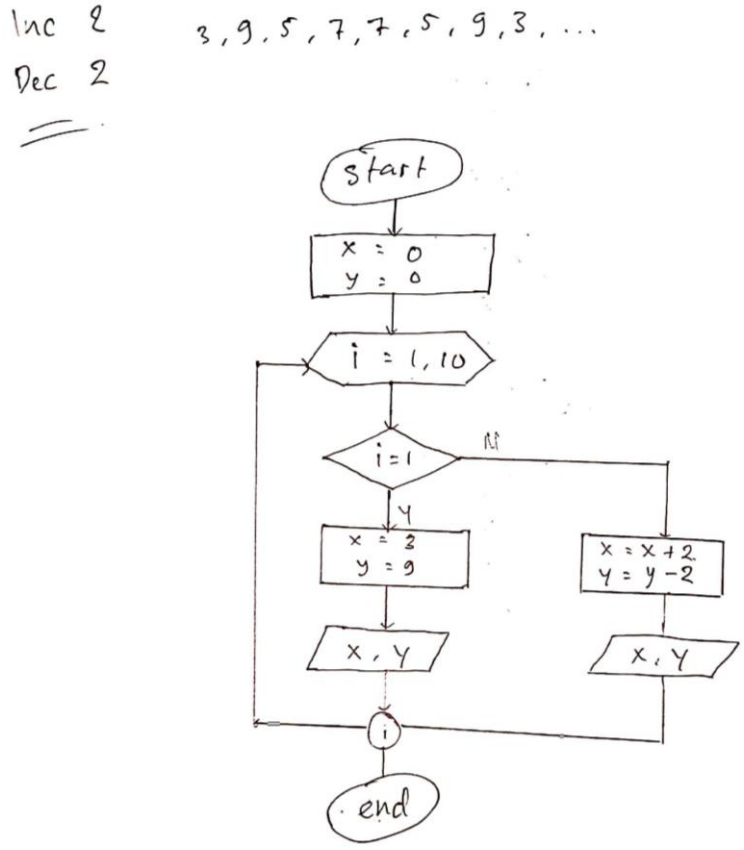
Flowchart



Analisis :

1. Mulai
2. Inisialisasi x=0 dan y=0
3. Inisialisasi I = 1 sampai 6
4. Inputkan nilai a
5. Cek kondisi apakah nilai a mod 2 = 0
6. Jika iya maka akan masuk ke proses x = x + a, lalu cetak nilai x
7. Jika tidak maka akan masuk ke proses y = y + a, lalu cetak nilai y
8. Ulangi Langkah 3 sampai i mencapai nilai ke 6
9. Selesai

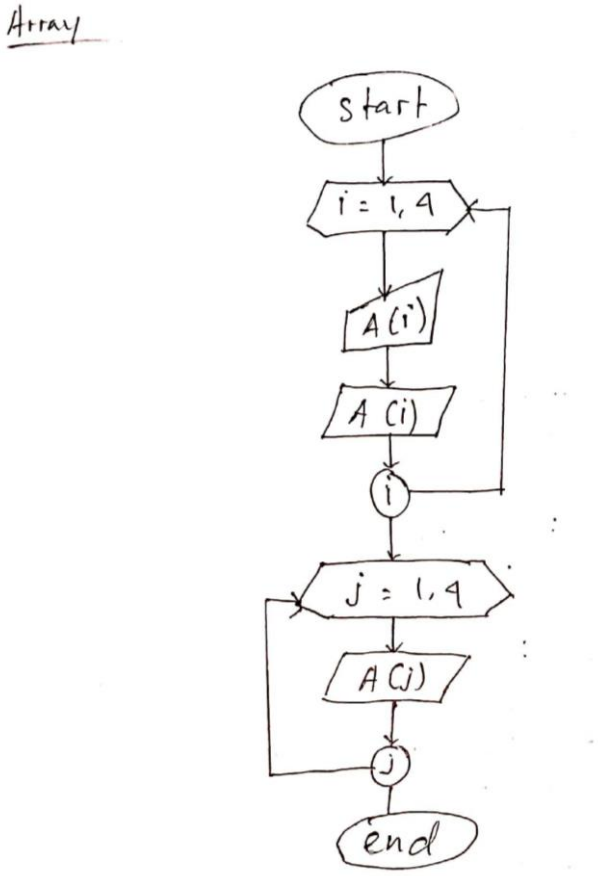
Flowchart



Analisis :

1. Mulai
2. Inisialisasi x=0 dan y=0
3. Isisialisasi i = 1 sampai 10
4. Cek kondisi apakah I = 1
5. Jika iya maka masuk ke proses x = 3 dan y = 9, lalu cetak x dan y
6. Jika iya maka masuk ke proses x = x + 2 dan y = y - 2, lalu cetak x dan y
7. Ulangi Langkah 3 sampai i mencapai nilai ke 10
8. Selesai

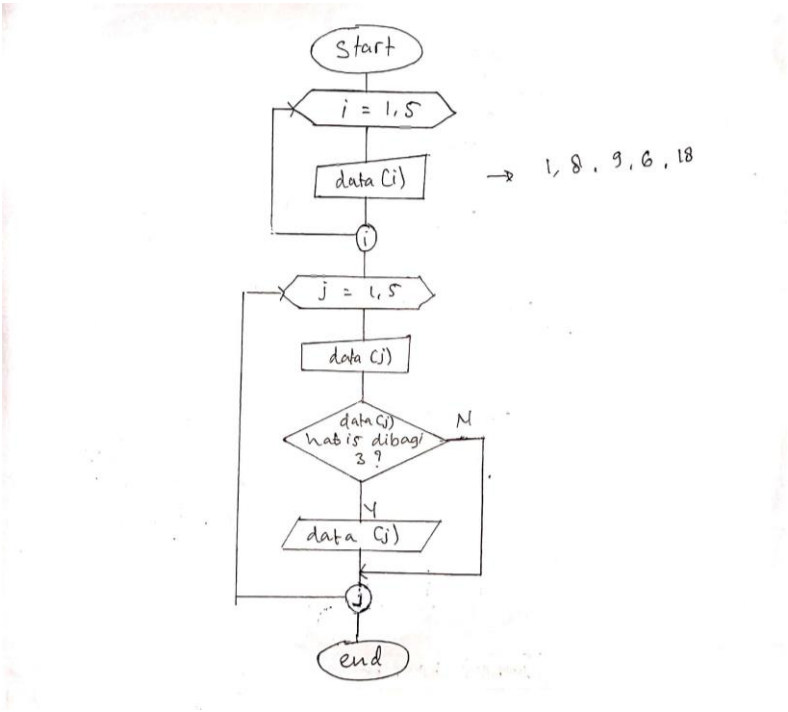
Flowchart



Analisis :

1. Mulai
2. Inisialisasi nilai i = 1 sampai 4
3. Inputkan variable A(i) berindex i
4. Cetak variable A(i) berindex i
5. Ulangi Langkah 2 sampai i mencapai nilai ke 4
6. Inisialisasi nilai j = 1 sampai 4
7. Inputkan variable A(j) berindex j
8. Cetak variable A(j) berindex i
9. Ulangi Langkah 6 sampai j mencapai nilai ke 4

Flowchart



Analisis :

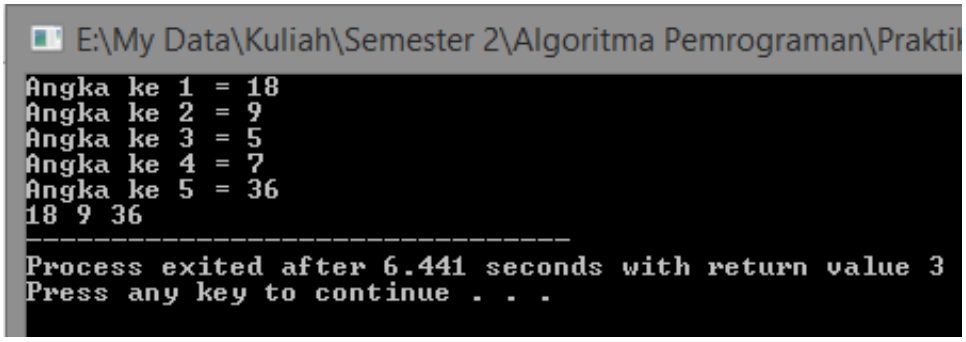
1. Memulai program
2. Dalam *preparation*, rentang i sebanyak 5
3. Inputkan nilai data(i)
4. Lakukan looping sebanyak rentang I, Kembali ke no 2
5. Setelah i terpenuhi ,melanjutkan ke tugas selanjutnya
6. Dalam *preparation*, diberikan j dengan rentang sebanyak 5
7. Inputkan nilai data(j)
8. Ditanyakan apakah input nilai j akan habis (memilki sisa nilai 0) jika dibagi 3
   Jika ya maka cetak nilai j
   Jika tidak maka tidak melakukan apapun
9. Lakukan looping sebanyak rentang j, Kembali ke nomor 6
10. Setelah j terpenuhi, program berakhir

Program dan Hasil

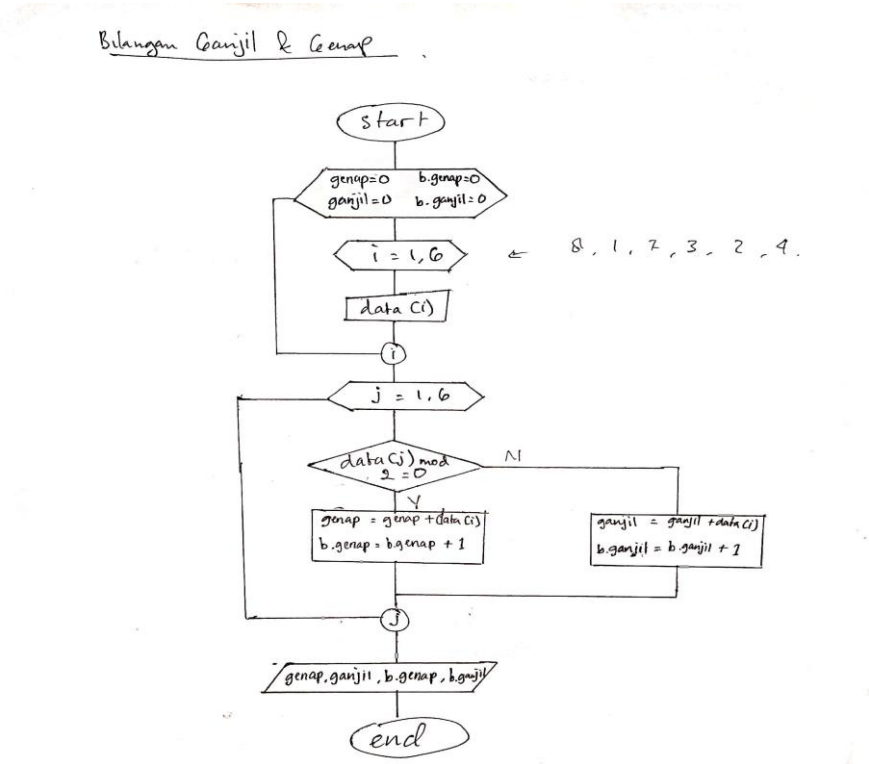
```c
#include <stdio.h>

int main(){
    int f[5];
    int i, j, x;

    for(i=0; i<5; i++) {
        printf("Angka ke %d = ", i+1);
        scanf("%d", &f[i]);
    }
    for (i=0;i<5; i++) {
        if (f[i]%3==0){
            printf("%d ", f[i]);
        }
        else{
        }
    }
}
```



Flowchart

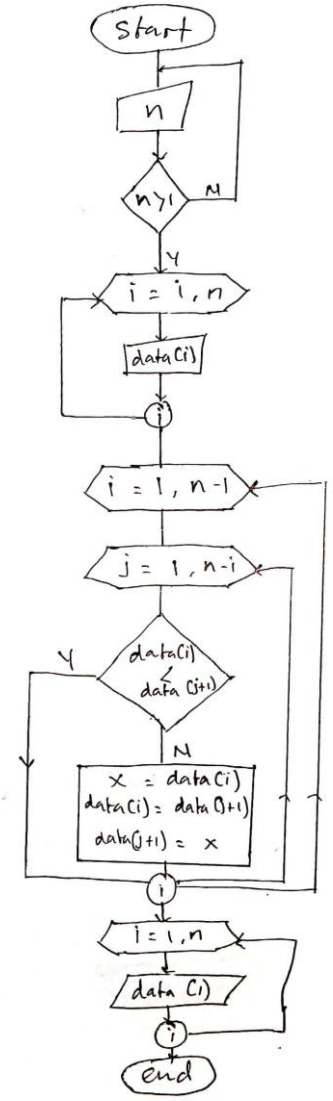


Analisis :

1. Memulai program
2. Dalam *preparation*, diberikan awalan sebagai berikut
   genap = 0
   ganjil = 0
   b. genap = 0
   b. ganjil = 0
3. Kemudian diberikan pula rentang i sebanyak 6
4. Inputkan nilai data(i)
5. Lakukan looping sebnayak rentang i, Kembali ke no 2
6. Setelah I terpenuhi, melanjutkan ke tugas selanjutnya
7. Dalam Dalam *preparation*, diberikan j dengan rentang sebanyak 6
8. Ditanyaakan apakah data(i) dibagi 2 memiliki sisa nilai 0 atau tidak
   Jika ya, maka dilakukan proses :
   genap = genap + data(i), dan
   b. genap = b. genap + 1

   Jika tidak, maka dilakukan proses :
   ganjil = ganjil + data(i), dan
   b. ganjil = b.ganjil + 1
9. Lakukan looping sebanyak rentang j, Kembali ke no 7
10. Setelah j terpenuhi, cetak genap, ganjil, b.genap, dan b.ganjil
11. Program berakhir

Flowchart



Analisis :

1. Memulai program
2. Inputkan nilai n
3. Ditanyakan, apakah nilai n bernilai lebih besar dari 1 atau tidak
   Jika tidak, maka kembali ke tugas no 2
   Jika ya, maka tugas dilanjutkan
4. Dalam preparation, diberikan nilai i memiliki rentang sebanyak n dengan awalan i
5. Inputkan data(i)
6. Lakukan looping sebanyak rentang n, Kembali ke no 4
7. Jika i terpenuhi, melanjutkan ke tugas selanjutnya
8. Dalam preparation, diberikan i memiliki rentang sebanyak n-1 dengan awalan 1

9. Dalam preparation pula, diberikan j memiliki rentang nilai sebanyak n-i dengan awalan 1
10. Ditanyakan apakah data(i) memiliki nilai lebih kecil dari data(j+1)
    Jika tidak maka akan dilakukan pengurutan angka(kecil ke besar)
    Jika ya maka tidak melakukan apapun, langsung menuju looping
11. Melakukan looping sebanyak rentang I, Kembali ke n0 8
12. Jika i sudah terpenuhi, melanjutkan ke tugas selanjutnya
13. Dalam preparation, diberikan i dengan rentang sebanyak n dan dengan awalan 1
14. Cetak data(i)
15. Lakukan looping sebanyak rentang i, kembali ke no 13
16. Jika i terpenuhi, program berakhir

Program dan hasil

```c
#include <stdio.h>

int main()
{
    int a;
    int b1;
    int b2;

    printf("Tukar bilangan\n\n");

    printf("Masukan bilangan pertama : ");
    scanf("%d",&b1);
    printf("Masukan bilangan kedua : ");
    scanf("%d",&b2);
    puts("");

    printf("Sebelum ditukar \n");
    printf("Nilai Bilangan pertama = %d \n", b1);
    printf("Nilai Bilangan kedua = %d \n", b2);
    puts("");

    a=b1;
    b1=b2;
    b2=a;

    printf("Sesudah ditukar \n");
    printf("Nilai Bilangan pertama = %d \n", b1);
    printf("Nilai Bilangan kedua = %d \n", b2);
    puts("");

    return 0;
}
```

```
E:\My Data\Kuliah\Semester 2\Algoritma Pemrograman\Prakti
Tukar bilangan

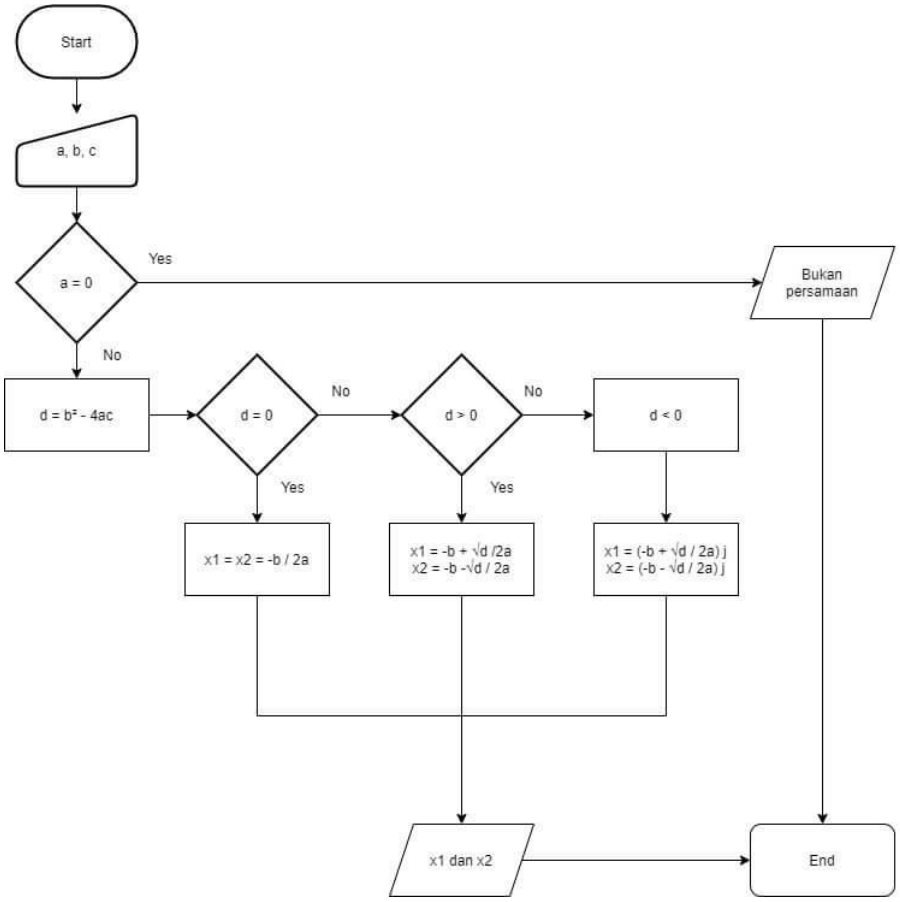
Masukan bilangan pertama : 74
Masukan bilangan kedua : 14

Sebelum ditukar
Nilai Bilangan pertama = 74
Nilai Bilangan kedua = 14

Sesudah ditukar
Nilai Bilangan pertama = 14
Nilai Bilangan kedua = 74


--------------------------------
Process exited after 9.335 seconds with return value 0
Press any key to continue . . .
```

Flowchart



Analisis :

1. Memulai program
2. Inputkan nilai a, b, dan c
3. Ditanyakan apakah a memiliki nilai 0 atau tidak
   Jika ya, maka cetak “Bukan persamaan”, dan program berakhir
   Jika tidak, maka melanjutkan ke proses pengoperasian matematika dengan rumus $d = b^2 - 4ac$
4. Cek kondisi Pada d = 0, jika iya maka lanjut ke proses x1 = x2 = -b/2*a. Jika tidak maka lanjutkan pada kodisi d > 0
5. Cek kondisi Pada d > 0, jika iya maka lanjut ke proses x1=(-b +√d)/2a dan x2=(-b-√d)/2a . Jika tidak maka lanjutkan pada proses d < 0
6. Pada proses d < 0, maka dihasilkan x1=(-b +√d)/2a j dan x2=(-b-√d)/2a j (x1, x2 adalah imajiner).
7. Cetak x1 dan x2
8. Program berakhir

Program dan hasil

```c
#include<stdio.h>
#include<math.h>

int main(){
    float a,b,c,D;

    double x1,x2;
    printf("masukkan nilai a= ");
    scanf("%f",&a);

    printf("masukkan nilai b= ");
    scanf("%f",&b);

    printf("masukkan nilai c= ");
    scanf("%f",&c);

    D=(b*b)-(4*a*c);

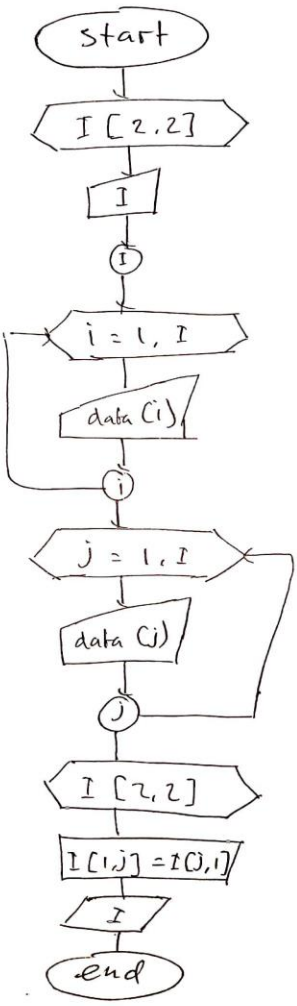
    if (D<0){
        printf("\nAkar-akar persamaan kuadrat imajiner\n");
    }else if (D>0){
        x1=(-b+(sqrt(D))/(2*a));
        x2=(-b-(sqrt(D))/(2*a));
        printf("\nAkar-akar persamaan kuadrat berbeda\n");
        printf("\nNilai x1=%.2f\n",x1);
        printf("Nilai x2=%.2f\n",x2);
    }else if (D==0){
        x1=(-b)/(2*a);
        x2=x1;
        printf("\nbukan persamaan");
    }
}
```

```
E:\My Data\Kuliah\Semester 2\Algoritma Pemrograman\Praktik
masukkan nilai a= 10
masukkan nilai b= 4
masukkan nilai c= 8

Akar-akar persamaan kuadrat imajiner

--------------------------------
Process exited after 4.919 seconds with return value 0
Press any key to continue . . .
```

Flowchart

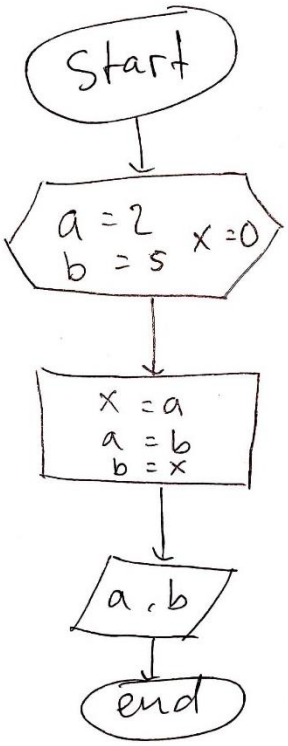


Program dan hasil

```
1   #include <stdio.h>
2
3   int main() {
4
5       int i, j, m, n;
6       int matriks [10][10];
7       int transpose [10][10];
8
9       printf("Jumlah baris = ");
10      scanf("%d", &m);
11
12      printf("Jumlah kolom = ");
13      scanf("%d", &n);
14
15      printf("Nilai matriks :\n");
16      for (i=0; i<m; i++){
17          for (j = 0; j<n; j++){
18              scanf("%d", &matriks[i][j]);
19          }
20      }
21      for (i=0; i<m; i++){
22          for (j = 0; j<n; j++){
23              transpose[j][i] = matriks [i][j];
24          }
25      }
26      printf("Nilai transpose matriks :\n");
27      for (i=0; i<n; i++){
28          for (j = 0; j<m; j++){
29              printf("%d\t", transpose[i][j]);
30          }
31      }
32  }
```

```
E:\My Data\Kuliah\Semester 2\Algoritma Pemrograman\Prakti
Jumlah baris = 2
Jumlah kolom = 2
Nilai matriks :
11      12
21      22
Nilai transpose matriks :
11      21      12      22
--------------------------------
Process exited after 11.42 seconds with return value 2
Press any key to continue . . .
```

Flowchart



Analisis :

1. Mulai
2. Inisialisasi a=2, b=5, dan x=0
3. Proses x=a, a=b, b=x
4. Cetak a dan b
5. Selesai

Program dan hasil

```
1   #include <stdio.h>
2
3   int main() {
4       int a=2, b=5, x=0;
5
6       x=a;
7       a=b;
8       b=x;
9
10      printf("nilai a=%d\nNilai b =%d", a,b);
11  }
```

```
C:\Users\DEIN S\Downloads\2.exe
nilai a=5
Nilai b =2
--------------------------------
Process exited after 0.1434 seconds with return value 20
Press any key to continue . . .
```

VI. KESIMPULAN

Berdasarkan Praktikum yang telah dilakukan dapat disimpulkan bahwa dapat disimpulkan bahwa dalam pembuatan program kita harus memperhatikan penggunaan huruf besar dan huruf kecil agar tidak terjadi error saat proses compile. Karena jika ada kesalahan sedikitpun akan terdeteksi pada program yang telah dibuat sebelumnya. Dengan membuat flowchar terlebih dahulu maka akan mempermudah untuk kita membuat suatu program

DAFTAR PUSTAKA

Andre. 2019. “Perulangan DO WHILE Bahasa C”. Dalam https://www.duniailkom.com/tutorial-belajar-c-perulangan-do-while-bahasa-c/ (5 Mei 2021)

Andre. 2019. “Perulangan For Bahasa C (Loop)”. Dalam https://www.duniailkom.com/tutorial-belajar-c-perulangan-for-bahasa-c/ (5 Mei 2021)

Andre. 2019. “Perulangan WHILE Bahasa C”. Dalam https://www.duniailkom.com/tutorial-belajar-c-perulangan-while-bahasa-c/ (5 Mei 2021)

Dalma, Afif. 2021. “Flowchart adalah – Pengertian, Jenis, Simbol, Contoh”. dalam https://dosenpintar.com/flowchart-adalah/ (5 Mei 2021)

Haidar, A. 2020. “ARRAY DI DALAM BAHASA PEMOGRAMAN C”.

Haidar, A. 2020. “FUNGSI DI DALAM BAHASA PEMOGRAMAN C,”.

Haidar, A. 2020. “PENGENALAN BAHASA C,”.

Haidar, A. 2020. STRUKTUR PEMILIHAN DALAM BAHASA C/C++,”

Informatikalogi. 2020. “Pengertian Flowchart dan Jenis-Jenisnya”. dalam https://informatikalogi.com/pengertian-flowchart-dan-jenis-jenisnya/#:~:text=DFD%20dan%20Flowchart-,1.,proses%20lainnya%20dalam%20suatu%20program. (5 Mei 2021)